Memory Hierarchy & Cache-Aware Algorithms

存储层次结构 与 缓存感知算法

现代算法讨论班 · 2026

目录

ROADMAP

01 **被忽略的代价**
RAM 模型的盲区

02 **存储层次结构**
从寄存器到磁盘

03 **理想缓存模型**
M、B 与 I/O 计数

04 **实例1：矩阵乘法**
优化再优化

05 **实例2：排序算法**
多路归并

06 **其他现代算法**
应用

01 · 被忽略的代价

一个简单的问题

遍历一个 $N \times N$ 数组?

考虑以下两则程序：

```
for(i=1;i≤n;i++)
    for(j=1;j≤n;j++)
        sum += A[i][j] ;
```

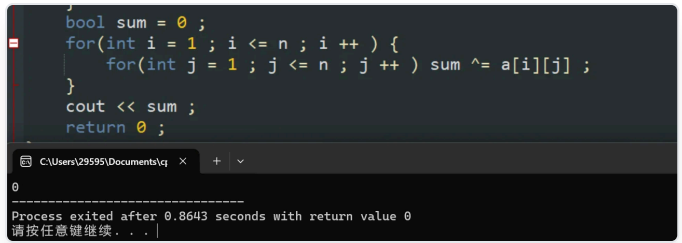
以及

```
for(j=1;j≤n;j++)
    for(i=1;i≤n;i++)
        sum += A[i][j] ;
```

通过正常的时间复杂度分析方法，显然两个程序复杂度都是 $O(n^2)$

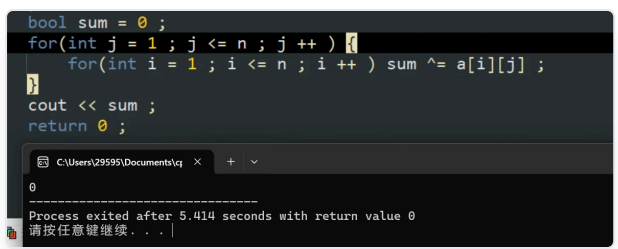
但实际运行效率真的如此吗?

按行遍历



> 耗时 0.86s

按列遍历



> 耗时 5.41s

理论分析复杂度一致的程序为何实际运行会有如此大差距？—— 我们需要更好的分析模型

RAM 模型：一个方便的虚构

模型假设：

- 内存是一个统一的字节数组
- 任意地址访问代价相同
- 不考虑缓存、预取、页表

代价

算法分析只看运算次数，不看数据移动。

对于**计算密集**任务，这足够好；对于**数据移动密集**任务，这是系统性偏差。

> 现代处理器的算术运算与 DRAM 访问之间，存在 **50–100 倍** 的延迟差距。RAM 模型把这个差距忽略了。

02 · 存储层次结构

从 CPU 到磁盘

层级	典型容量	典型延迟	管理者
寄存器	数百字节	< 1 cycle	编译器
L1 Cache	32–64 KB	~4 cycles	硬件
L2 Cache	256 KB–2 MB	~12 cycles	硬件
L3 Cache	数 MB–数十 MB	~40 cycles	硬件
主存（DRAM）	GB–TB	~200 cycles	OS
SSD / 磁盘	TB+	10^5–10^6 cycles	OS

> 越靠近 CPU，越快、越小、越贵。。

03 · 理想缓存模型

External Memory / Ideal-Cache 模型

模型参数

- 快速内存容量：$M(\mathrm{KB}, \mathrm{MB}, \mathrm{GB}级别)$
- 数据块大小：B（$32, 64, 128, 1024B$）
- 外存：无限大
- 定义一次 I/O：搬入或写回**一个块**

代价度量

算法代价 = **I/O 次数**，而不是运算次数。

CPU 直接访问快速内存中的数据以及进行运算视为“免费”，但把数据从外存搬到快速内存或内存写入外存要按块计数。

Ideal-Cache的两条假设

全相联

任意一个块可以放在缓存的**任意位置**，不受组相联映射的限制。

最优替换

缓存满时，替换**未来最晚才会被用到**的块，即 Belady 的 MIN 策略。

> 在最优替换下，任意算法 A 与最优离线算法 （OPT） 的 I/O 次数满足 $Q_A \leq C \cdot Q_{OPT}$，其中 C 是与 M、B 无关的常数。进一步地，LRU 与最优替换的 I/O 差距也在**常数因子**以内。因此，理想模型下证明的**渐近 I/O 最优性，对真实硬件依然成立**。

在这些假设下，一个程序如果具有**时间局部性**与**空间局部性**将会效率更高。

说人话是，访问过的元素不久后会被再次访问 以及 被访问的元素地址相邻的元素不久后也会被访问

04 · 实例1：矩阵乘法

朴素实现

设有 n 阶方阵 A, B, C，矩阵乘法 $C = A \times B$ 定义为：

$$c_{i,j} = \sum_{k=1}^{n} a_{i,k} \times b_{k,j}$$

直接按照定义实现三重循环，最内层沿 k 遍历。

```
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      C[i][j] += A[i][k] * B[k][j] ;
```

我们发现对于 B 这个二维数组来说，它每次都在被按列遍历，没有做到地址连续访问，因此效率不高

用 I/O 计数的方法分析时间复杂度：$O(n^3)$

一个小优化：交换内层循环

考虑把 j 循环移到内层，使 $B[k][j]$ 变为连续访问。

```
for (i = 0; i < N; i++)
  for (k = 0; k < N; k++) {
    for (j = 0; j < N; j++)
      C[i][j] += A[i][k] * B[k][j];
  }
```

由于三个数组下标都被连续访问，每 B 个元素都可以被 "打包" 读入内存中，复杂度为 $O(\frac{n^3}{B})$

那么是否还有更好的做法呢？再分析以下代码中每个数组的遍历情况：

A：当 $A[i][k]$ 被遍历到搬入内存之后，所有涉及到它的运算会被**一次解决**完，未来不再需要搬运

C：对最内层来说，随着 k 的循环我们会把第 i 行重复遍历 N 次，这意味这当 $M < N$ 时，一定会存在某个块被多次地写入\写出，造成浪费。

B和C是同理的。

究其根本是这个算法的局部性不足，同一个元素会在多个时间点被写入写出

Cache-Aware 分块

把矩阵切成 $s \times s$ 的子块，使三个子块能同时装入缓存。

选择

$$s = O(\sqrt{M})$$

然后对 $\frac{N}{s} \times \frac{N}{s}$ 的块矩阵执行三重循环。

分析复杂度，得到：

$$\frac{s^2}{B} \times (\frac{N}{s})^3 = O\left(\frac{N^3}{B\sqrt{M}}\right)$$

> 这是矩阵乘法的**I/O 下界**量级。分块把工作集压到缓存容量以内，使每一行缓存在被逐出之前被尽可能多地复用。

如果未知 M，未知 B 时怎么办?

Cache-Oblivious 算法不知道 M 和 B 的值，也不针对某一特定缓存层级调参。

做法很简单：每次将矩阵四等分，递归计算小矩阵的乘积。

当递归到的子矩阵小到一定程度时，即可一次写入写出，不再消耗代价。

```
void matmul(A, B, C, N) {
    if (N ≤ BASE) {            // 基础情形：小到自然装进缓存
        naive_matmul(A, B, C, N);
        return;
    }
    // 切成 8 个子问题，递归
    for (i = 0; i < 2; i++)
        for (j = 0; j < 2; j++)
            for (k = 0; k < 2; k++)
                matmul(A[i][k], B[k][j], C[i][j], N/2);
}
```

ps：这里的 BASE 仅仅是一个较小的常数(取成1都是可以的)，完全不依赖知道M或B的取值，目的是为了减小递归常数

分析一下 I/O 复杂度：

$$T(N) = 8T(\frac{N}{2}) + O(\frac{N^2}{B})$$

边界：当 $N < \sqrt{M}$ 时，$T(N) = 0$，不再需要 IO

经过分析，最终复杂度同样为：$O\left(\frac{N^3}{B\sqrt{M}}\right)$

> 与 cache-aware 分块**渐近相同**，但不需要提前知道任何硬件参数。递归树的每一层“自动适配”某一级缓存。

递归常数大！

对比

Cache-Aware

先测量 M，再选块大小 $s = \Theta(\sqrt{M})$。

换一台机器，需要重新调参。

Cache-Oblivious

不测量，不调参。递归直到子问题自然落入缓存。

同一份代码在不同的缓存 L1、L2、L3 上同时获得渐近最优的 I/O。

> 代价是常数因子和实现复杂度。实践中通常会在递归到某个“叶子”规模时停止，接入手工微内核。

06 · 实例2：排序

问题很简单：

- N 个元素，存放在外存。
- 内存容量 M，单次搬运块大小为 B。
- 目标：将 N 个元素排序后写回外存。

最常见的做法是 **归并排序**，RAM模型下复杂度为 $O(n \log_2 n)$

二路归并

直接照搬做法，分析复杂度：

$T(N) = 2T(N/2) + O(\frac{N}{B})$

边界：当 $N < M$ 时，$T(N) = 0$

复杂度为 $O(\frac{N}{B} \log_2 \frac{N}{M})$

多路归并

k 路归并：每次从 k 个有序序列中各取一个块放入缓存，合并后写回。

缓存约束

$$M \geq (k+1)B$$

因此 $k \leq \Theta(M/B)$。

总 I/O

$$T(N) = kT(N/k) + O(\frac{N}{B})$$

得到

$$O(\frac{N}{B}\log_k \frac{N}{M})$$

> 取 $k = \Theta(M/B)$ 时，归并排序达到 $O\left(\frac{n}{B}\log_{M/B}\frac{n}{M}\right)$，这是比较排序在 Ideal-Cache 模型下的 **I/O 最优**量级。

对比

标准 2 路归并的 I/O 是 $O\left(\frac{n}{B}\log\frac{n}{M}\right)$。多路归并把底数从 2 提高到 M/B，减少了递归层数，从而降低了总 I/O。

2 路归并

每层把问题减半，需要 $\log(n/M)$ 层合并。

k 路归并

每层把问题缩小 k 倍，层数降为 $\log_k(n/M)$。

为什么 RAM 模型下不采取 k 路归并?

07 · 其他现代算法

PACO（2026）

PACO 是一个**完全 cache-oblivious 的并行 FFT**。它用递归的 Cooley–Tukey 分解，调度器在每一层做平衡的“数字对齐”切分，不依赖任何缓存参数。关键结果：在 tall-cache 假设 $\left(M = \Omega(B^2)\right)$ 下，局部 FFT 的缓存缺失为

$$Q_{\text{local}}(n, s) = O\left(\frac{ns}{B}\left(1 + \log_M n\right)\right)$$

并且算法本身**从不检测 (M)**——“缓存能装下”的临界点是纯分析用的边界。实验上，它在昇腾 910 NPU 上比厂商 FFT 快 (1.53\times)，在跨平台对比中略快于 NVIDIA A800 上的 cuFFT。

这就是递归四分矩阵乘法的“同款结构”——递归切分让子问题在所有尺度上自动“落进”某一层缓存，算法不知道 (M,B)，却获得了跨层最优的 I/O 行为。

LSM-tree Compaction（2025–2026）

LSM-tree（RocksDB、LevelDB 等）的 compaction 本质上是**多路归并排序**：把 (k) 个有序 SSTable 合并成更大的有序文件。归并路数 (k)、每层容量、SSTable 大小，共同决定**写放大**和**读放大**的权衡。C2LSM（2026）专门形式化了“每层容量、SSTable 大小、compaction 时间”之间的关系，用来优化配置参数。Mosaic（2026）则把 compaction 数据流解耦为前台和后台独立路径，在限定写放大的前提下并行化子树 compaction。

多路归并的 I/O 是

$$\Theta\left(\frac{N}{B}\log_{M/B}\frac{N}{B}\right)$$

LSM-tree 的层数、每层归并路数、缓冲区大小，就是 (M/B) 和 $(\log_{M/B})$ 的工程化身。归并路数选小了，趟数多、写放大高；选大了，单趟 I/O 和内存压力上升。

GPU 稀疏矩阵乘法（2025–2026）

GPU 上的稀疏矩阵乘法（SpMM/SpMV）面临和 CPU 分块矩阵乘法**同构**的问题：稀疏矩阵的非零分布导致访问不连续，稠密操作数 B 的**时间局部性差**、**非合并内存事务**多。近年的工作直接以“cache-friendly / cache-aware”为设计目标：

- **C3ache**（MICRO 2025）提出分层 cache-centric 计算架构 + 混合数据流，在 GPGPU 上相对 CUDA Core 平均加速 (7.22\times)，EDP 降低 (42.4\times)；
- **CB-Sparse**（2025）用 cache-friendly 的数据聚合算法，缓存命中率最高提升 **47.8%**；
- **VCSR-MM**（ISPASS 2026）用 Stream Tiling **消除冗余 L2 缺失**、减少非合并全局事务。

这些工作本质上都在做“显式分块 + 工作集控制”。它们的性能差异直接来自**子块能否驻留某一层缓存**——这正是 $(s = \Theta(\sqrt{M}))$ 和常数因子分析在 GPU 内存层次上的现实版本。

谢谢

Memory Hierarchy & Cache-Aware Algorithms